
To Forge or Not to Forge: Predicting Task-Level Fine-Tuning Gains from Ten-Example Pilots

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Fine-tuning a small specialist is cheap, but only after one decides which tasks
2 to train. Under one frozen protocol (Qwen2.5-1.5B-Instruct, fixed LoRA, greedy
3 pass@1) we pair a ten-example probe with a full fine-tune on 61 tasks (53/61 go,
4 87% in this regime). The probe *rank*s gain (Spearman 0.755, CI [0.595, 0.864])
5 and a budget of $K=20$ full slots chosen by Δ_{pilot} captures 72% of all positive-gain
6 mass (random 33%, oracle 74%). The registered binary LOTO, which included
7 a post-decision feature, is negative (AUC 0.755, CI [0.500, 0.929] covers 0.5).
8 After removing leakage the revision LOTO is 0.844 [0.736, 0.939] (permutation
9 $p=0.001$), but a perfect gate saves only 2.4% of positive-gain mass versus always
10 forging. Instruction embeddings are chance (AUC 0.51); 23/61 tasks have base
11 pass@1 = 0 (format unlock). The table was produced by an autonomous discover-
12 pilot-decide-train-validate harness.

13 **Keywords:** fine-tuning, evaluation, small language models

14 1 Autonomous research result

15 1.1 Hypothesis

16 Under a single frozen LoRA recipe on a 1.5B instruction-tuned base, can a ten-example probe decide
17 *which* tasks to fully fine-tune, and is that decision a binary gate or a ranking for a budget of K slots?
18 We pre-register a binary label $\text{go} \Leftrightarrow \Delta_{\text{full}} > 0$ and a secondary ranking endpoint $\rho(\Delta_{\text{pilot}}, \Delta_{\text{full}})$,
19 with the honesty clause that a non-significant AUC is published as a negative. A later, declared
20 revision (v2) asks the same questions after removing post-decision features from the classifier, and
21 quantifies the *utility* of a perfect gate versus always forging.

22 1.2 Method

23 **Protocol and sample.** Base model Qwen2.5-1.5B-Instruct [Qwen Team, 2024], weights pinned
24 at birth; LoRA rank 16, $\alpha = 32$; sequence 4096; effective batch 16; pilot 10×100 ; full cap 8000
25 / 3 epochs; eval 200, greedy. pass@8 ($k=8$) is a signal (1600 generations per task), never the
26 label. The sealed sample is $n=61$ (11 literature-layer + 50 SuperNI, seed 20260822): 53 registered
27 go / 8 no-go (87%). BIRD and APPS are out, excluded for logistics, before any label
28 existed. All numbers below that are not marked “registered” come from ANALYSIS_RESULTS_v2
29 (CPU revision). Registered AUC/CI remain the v1 sealed row.

30 **Predictors.** The *new main* classifier is leave-one-task-out logistic regression on the pre-decision
31 vector only: Δ_{pilot} , base pass@1, base pass@8, headroom, pilot-loss descriptors, format-
32 compliance of the *base* split, and train_n. gen_len.full.median is reported as a post-hoc oracle

upper bound and is *not* used for decisions. The registered 12-feature vector (which included that oracle and full_n) is kept as the registered row. Intervals are task-level bootstrap 95% CIs (seed 20260820, $B=1000$). A label-permutation test re-runs full LOTO $N=1000$ times (seed 20260826). We also report L2-CV LOTO, a 3-feature model (Δ_{pilot} , base_pass1, headroom), and a no-pass@8 feature set. Instruction embeddings (MiniLM, CPU) plus SuperNI Categories/Source are the semantic control; the original four scalars are the metadata control. McNemar exact tests on paired eval jsonl induce a three-way label (go / no-go / undetermined) as a revision-period analysis; the registered label stays $\Delta_{\text{full}} > 0$. Sensitivity (b) (drop EM-floor rows) is computed but *not* used as supporting evidence (wide CI).

Related work (lite). Training-free transferability estimators (LogME, LEEP, TransRate) score a frozen representation against target labels without fine-tuning [You et al., 2021, Nguyen et al., 2020, Huang et al., 2021]. We did *not* run them: they select among source models, whereas we select among *tasks* given one frozen recipe. That is a limitation and a listed future comparison. Multi-fidelity methods (successive halving, Hyperband, learning-curve extrapolation) [Jamieson and Talwalkar, 2016, Li et al., 2018] treat cheap runs as a fidelity axis. Our probe is a *single* fidelity point; a fidelity scan is future work.

1.3 Evidence

Finding 1: forging is the default in this regime. 53 of 61 tasks (87%) have $\Delta_{\text{full}} > 0$. This is Qwen2.5-1.5B-Instruct, one LoRA recipe, greedy pass@1, gate $\Delta_{\text{full}} > 0$ — not a claim about 7B models or full FT. 23 of 61 tasks have base_pass1 = 0; most of those gains are format unlock (the probe teaches the extractor), not new reasoning.

Finding 2: the probe ranks gain, and the ranking has budget value. Spearman $\rho(\Delta_{\text{pilot}}, \Delta_{\text{full}}) = 0.755$, CI [0.595, 0.864] (registered secondary; excludes 0). Partial ρ controlling for base_pass1 is 0.832: the ranking is not a restatement of base accuracy. Figure 1 is the operational plot. Ranking by Δ_{pilot} and spending a budget of K full fine-tunes captures 0.42 / 0.72 / 0.83 of all positive-gain mass at $K=10, 20, 30$, against random 0.16 / 0.33 / 0.49 and oracle 0.45 / 0.74 / 0.90. At $K=20$ the probe is within three points of oracle. 38% of tasks have $|\Delta_{\text{pilot}}| \leq 0.02$ (a dumb probe); the ranking still works because the mass sits in the tails. Figure 2 shows the same pairing; GSM8K/MATH sit in Q3 and the EM-floor quartet sits at the origin.

Finding 3: the binary gate, three ways. *Registered row.* LOTO on the original 12-feature vector (including post-decision gen_len.full.median) has AUC 0.755, CI [0.500, 0.929], covering 0.5 — a negative under the honesty clause. (The point 0.7547 and Spearman 0.7553 both print as 0.755; that is a rounding coincidence, not one statistic.) *Revision main row.* After dropping post-decision features, LOTO AUC = 0.844, CI [0.736, 0.939], permutation $p=0.001$ ($N=1000$); bootstrap and permutation *agree* (both reject chance). L2-CV is 0.875 [0.763, 0.960]; the 3-feature model is 0.767 [0.649, 0.870]; dropping pass@8 still yields 0.837 [0.721, 0.930]. The v1 negative is therefore better read as *estimator-plus-leakage*, not as “no signal.” *Utility.* A perfect gate’s upside versus always-forging is $|\sum \text{negative } \Delta| / \sum \text{positive } \Delta = 0.024$. Net always-forge is +16.6 pass points; oracle-forge is +17.0. McNemar three-way labels are 45 go / 2 no-go / 14 undetermined ($\alpha=0.05$, uncorrected). With two significant negatives the determined-only AUC is not a serious gate estimate. Figure 3 is the pre-decision ROC.

Finding 4: you still have to run the probe. Instruction-sentence embeddings plus SuperNI category/source have AUC 0.512 [0.255, 0.746] — chance. The old four-scalar “text-only” AUC 0.309 is a *metadata* control whose eval_n is constantly 200; it is not evidence that description features run below chance in any semantic sense. Single-feature steps_to_0.01 is 0.748 [0.585, 0.881] (does not cover 0.5; five tests, *not* multiplicity-corrected). gen_len.full.median is 0.818 [0.528, 0.978] and is post-hoc.

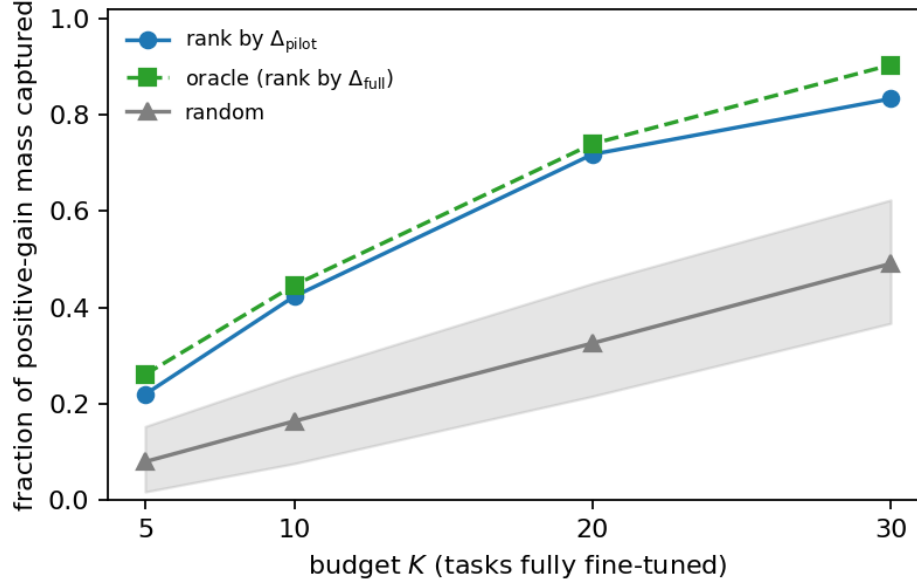


Figure 1: Budget allocation. K full fine-tunes, fraction of positive Δ_{full} mass captured.

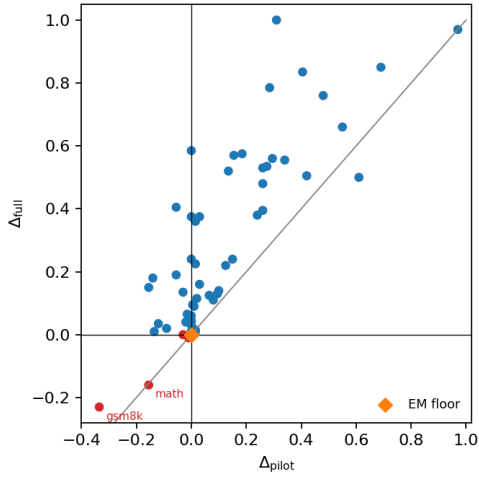


Figure 2: Pilot vs. full, cropped. Diamonds: EM-floor rows.

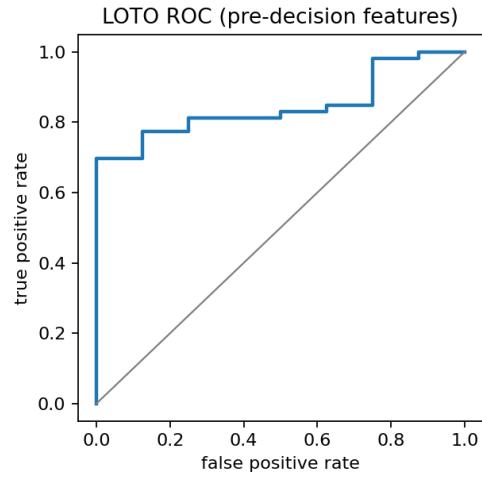


Figure 3: LOTO ROC, pre-decision features.

Table 1: Classifiers on the registered go label. v2 is pre-decision. Registered 12-feat is the honesty-clause row.

Model	AUC	95% CI	covers 0.5
Registered 12-feat (v1)	0.755	[0.500, 0.929]	yes
Pre-decision LOTO (v2 main)	0.844	[0.736, 0.939]	no
L2-CV pre-decision	0.875	[0.763, 0.960]	no
3-feat	0.767	[0.649, 0.870]	no
No pass@8	0.837	[0.721, 0.930]	no
Semantic (MiniLM)	0.512	[0.255, 0.746]	yes
Metadata (4 scalars)	0.309	[0.051, 0.675]	yes

80 Median wall $(S2+S3)/(S4+S5) = 0.17$ (IQR $[0.11, 0.36]$); including S1, 0.53 (IQR $[0.35, 1.19]$).
81 pass@8 is 1600 generations per task and is not required for the gate (no-pass@8 row).

82 1.4 Primary Result

83 In this regime the probe is a *ranker with measured budget value*, not a stoplight worth building
84 operations around. Spending $K=20$ full slots by Δ_{pilot} harvests 72% of all positive gain, near the
85 oracle 74%. The registered binary endpoint was negative once post-decision features were in the
86 vector; after they are removed the LOTO gate is significant — and still almost useless as a *decision*,
87 because a perfect gate saves 2.4% of positive-gain mass relative to always forging. The 87% base
88 rate, 23/61 zero-base format unlocks, and McNemar’s two significant negatives are the same fact:
89 no-go is scarce. Instruction text does not replace the probe (semantic AUC 0.51). We do not reopen
90 BIRD/APPS, do not move the registered go threshold, and do not treat sensitivity (b) as evidence.

2 System Design and Meta-Analysis

2.1 Agent and Harness

We ran the study with two coupled pieces: a frozen Python harness that executes one task from package load through a sealed `metrics.json`, and an LLM coding agent that packed those tasks, drove the remote GPU worker, diagnosed failures, ran the pre-registered analysis, and drafted this manuscript. The experimental model inside the harness is Qwen2.5-1.5B-Instruct at a pinned revision; the coding agent’s weights are omitted here for double-blind review and are named in the disclosure for the chairs.

The harness is a single-task state machine S0–S6: load and hash-check the package; greedy-evaluate the unadapted base; train a 10-example LoRA probe; evaluate the probe; train the full LoRA; evaluate the full adapter; seal metrics (systems block included). Resume is journal-boundary only: a killed full-training stage restarts at S4 with S0–S3 kept. The agent never edits the protocol file. It may add scripts, isolate a failed run directory, or stop and write a report.

Figure 4 is the loop that produced the sealed $n=61$ table: a task factory writes frozen packages; a serial queue feeds the S0–S6 runner; gates mark PARTIAL or over_budget without inventing labels; incremental then full harvest brings artifacts back; a pre-registered script seals the analysis.

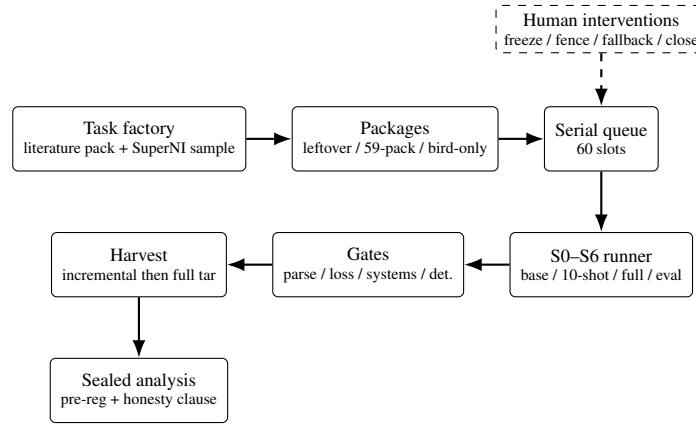


Figure 4: Factory → frozen packages → serial queue → S0–S6 runner → gates → harvest → sealed analysis. Dashed box: human intervention points (freeze, fence, fallback, close).

2.2 Tools

Compute was one 48GB-class GPU worker and one serial queue. The worker ran the harness under a terminal multiplexer; the agent supervised it with scheduled probes of `journal.jsonl`, `STATUS`, and `metrics.json`, because a remote job does not wake the session when it ends. Local tools were git on a single mainline (small commits, no force-push), an append-only ledger, pytest, and SHA-256 manifests. Execution-based verifiers (GSM8K, WinoGrande, Spider, later MATH/MBPP/DROP/TyDiQA/ARC/HellaSwag/PIQA and a SuperNI exact-match family) expose a uniform `verify(output, reference) -> {pass, parsed, note}`; unparseable is parsed is None, never a silent wrong. Each verifier has mutation tests: gold must pass, a known-wrong answer must fail, format variants of a correct answer must pass, garbage must unparse. GSM8K accepts ##### 1,000 as 1000 and rejects a hash without a number; WinoGrande will not treat the bare English article “a” as choice A; Spider compares execution multisets and treats a 30s timeout as fail with the exception type in note. The factory pins source revisions at pack time (dataset SHAs in the package `task.json`) and writes a per-package `MANIFEST.sha256`. SuperNI clones were frozen at 55a36563... before ids were committed; English+size filter left 890 surviving tasks, then the seed-20260822 draw of 50 with a Source[0] cap of 3. Code execution is a subprocess with a 10s cap, no network, and a 2 GiB virtual-memory limit.

124 2.3 Research Loop

125 The loop was not an open-ended search over architectures. It was discover (pack a frozen task), pilot
126 (10-shot LoRA), decide (record Δ_{pilot} as a signal, not a gate), train (full LoRA), validate (greedy
127 pass@1), then stop. The scientific hypothesis — whether that probe classifies go $\Leftrightarrow \Delta_{\text{full}} > 0$ —
128 was written down before the remaining production runs.

129 Protocol freeze came first. After three smoke runs the operator stopped a proposed `protocol_v3`:
130 the pilot 10×100 reaching loss $\sim 10^{-5}$ is recitation, and recitation is the probe, not a bug.
131 `protocol_v2.yaml` was never edited after that freeze; any later exception is a run-level flag, not a
132 silent yaml change.

133 Sample-frame correction came next. An earlier count treated 46 retestable *pool rows* as 46 tasks.
134 Deduped literature-layer tasks are 13 (3 already sealed, 10 new) plus 50 SuperNI tasks drawn by a
135 pre-registered English-and-size filter (seed 20260822, `Source[0] ≤ 3`). Intended $n=63$. A second
136 GPU worker was cancelled; the 60 new slots ran as one serial list. BIRD was packed later as its own
137 package; the 59-task package did not wait. Mini-Dev was never used.

138 The serial queue started 2026-08-22T09:34Z and drained 60/60 at 2026-08-24T00:22Z on that one
139 GPU (39 h wall). Of those 60 slots, 55 sealed `STATUS=ok` on the 59-task package; the rest were PAR-
140 TIAL or `over_budget` (TyDiQA and task419 S4 OOM, ARC-Easy unparseable at 5% with metrics,
141 BIRD S0 field error, APPS 3 h S1-only). Three earlier smoke runs (GSM8K, WinoGrande, Spider)
142 already counted as production data under the freeze. A per-task 3 h wall marks `over_budget`; gates
143 run after S6 and do not block the next task. Determinism uses a 200-row rerun on list items 1 and
144 every 10th, otherwise 30 rows. Incremental harvest pulled a 10-directory tar after every tenth GPU
145 completion (adapters omitted). Failed directories are isolated, not deleted. A four-task rerun wave
146 (declared before any AUC) then ran 2026-08-25T06:19Z–13:11Z: BIRD dual-sha checker, TyDiQA
147 and SuperNI task419 with a disclosed 1×16 OOM fallback (effective batch still 16), APPS at an
148 8 h ops fence. Analysis was sealed the same day at $n=61$ under the honesty clause. No AUC was
149 computed until that script ran.

150 2.4 Human Interventions

151 Human interventions were discrete, dated, and written into the ledger before the next GPU step. We
152 list the ones that changed what the sealed table contains, including the agent’s own misreports.

153 **False-negative connectivity.** The agent reported that the GPU worker was unreachable. A follow-
154 up over an already-open multiplexed session returned ok. The cause was the agent’s tool layer
155 refusing the remote-shell call, not a host or authentication failure. Production started after that
156 correction, not after a re-provision. That is an agent error in the log, not a cluster outage.

157 **Procedure slip, then a STOP.** Gate 1 (WinoGrande, then Spider) was started before Gate 0’s five-
158 eye table had been accepted. The operator halted Spider at the S4 boundary (`SIGTERM`; S0–S3
159 kept), kept the WinoGrande directory, and only then passed Gate 0. Spider later resumed from that
160 journal boundary and sealed. The STOP did what a comment in chat cannot: it prevented a protocol
161 decision from being made on a moving GPU.

162 **Format overwrite, not EOS.** GSM8K returned $\Delta_{\text{full}} = -0.230$ with unparseable 0/200 and a
163 deterministic greedy rerun. A truncation/EOS reading does not fit those numbers. Base format-
164 compliance was ##### 58 / last-number 142 / none 0; after the probe and after full training the slice
165 was ##### 200/200. The operator’s Gate 0 ruling — format overwrite with reasoning collapse — is
166 the one we keep. The derived `format_compliance` field was backfilled into metrics and did not
167 bump the protocol version.

168 **Schema lag on BIRD, failed closed.** The first BIRD slot died in S0 on `KeyError: sha256`:
169 the checker wanted a single `source.sha256`; the package stored `train_zip_sha256` and
170 `dev_zip_sha256`. No metrics were written. The directory was isolated. The checker was aligned to
171 the dual-sha fields; the bird-only package was not rebuilt. A later slot passed S0, then died of CUDA
172 OOM at the first probe backward pass (S1 pass@1 0.085 / pass@8 0.23; still no `metrics.json`).
173 We did not invent a SQL label to close the queue.

174 **Wall as an ops fence, not a measurement.** APPS hit the 3h cap in S1 with no metrics
175 (over_budget). The 8h rerun was declared before any APPS number existed. It finished S1 at
176 base pass@1 0.0 / pass@8 0.005 and died of CUDA OOM at full-training step 22/495 — still no
177 Δ_{full} . The floor is a missing-negative footnote, not a row.

178 **OOM fallback, two tasks only.** TyDiQA and SuperNI task419 died of CUDA OOM in S4 on
179 the first pass. A run-level 1×16 fallback (effective batch 16, yaml untouched, flag in STATUS and
180 metrics) was declared before those reruns, and only those two task ids may carry the flag. Both
181 resealed: $\Delta_{\text{full}} + 0.530 / +0.505$. ARC-Easy stayed PARTIAL (unparseable 10/200 = 5%, metrics
182 present) and entered the main sample; sensitivity (a) drops it.

183 **Campaign close.** BIRD and APPS stay out: excluded for logistics, before any label
184 existed. The go threshold, the protocol file, and the honesty clause were not reopened after seeing
185 AUC.

186 **What the operator always owns.** The following decisions never moved to the agent, even when
187 the agent drafted the patch: freeze or not freeze the protocol file; keep or replace the 10×100 probe;
188 the honesty clause and the go threshold; whether Mini-Dev may substitute for BIRD; the exact
189 exclude sentence and the rule that it may be used only before a label exists; whether a PARTIAL
190 with metrics (ARC-Easy) enters the sample; whether the pod may be killed; the disclosure boxes in
191 this PDF. The agent owns packing, the unattended queue, harvest SHA checks, five-eye tables, and
192 writing PARTIAL instead of a guessed Δ . When those two columns mixed — the SSH misreport,
193 Gate 1 started early — the log names the mix rather than rewriting it.

194 2.5 Verification

195 Four checks sit under every sealed row, and a fifth sits under the paper’s numbers.

196 Gates (automatic, do not block the queue): base unparseable < 5%; pilot loss must fall; the sys-
197 tems block must be complete (torch, transformers, CUDA, driver, GPU name, base revision, seeds,
198 dry_run=false); a greedy rerun of the unadapted base must match per-id pass (full 200 on list
199 items 1 and every 10th; otherwise 30 rows; seed 20260820). Failure writes STATUS=PARTIAL.
200 Smoke three-task five-eye was all green (GSM8K/WinoGrande unparseable 0/200, Spider 1/200 =
201 0.5%; rerun mismatch 0).

202 Packages carry MANIFEST.sha256. Incremental harvest tars omit adapters and were SHA-
203 checked against the worker; the full archive (adapters and isolated dirs included) has sha256
204 71309f7c...f4028f7d. Figures in Part 1 are redrawn from a frozen plot payload, not from a second
205 analysis.

206 The analysis contract — go iff $\Delta_{\text{full}} > 0$, task-level LOTO logistic, Spearman on the two deltas,
207 text-only control, bootstrap 95% CI, “AUC non-significant = publish negative” — was committed
208 before the remaining 60 GPU slots. The coding agent drafted this Part 2 from that ledger and those
209 reports. A human author has to walk every claim before the disclosure boxes below can be ticked;
210 they are unchecked in this draft.

211 2.6 Significance of result

212 The qualifying result is a 61-row, execution-based ground-truth table that did not exist before the
213 harness ran, plus a pre-registered reading of it: forging is the default (53/61), the 10-shot probe
214 ranks gain, and the binary gate is not established. Small-model post-training spends real GPU time
215 on tasks that look like GSM8K (negative transfer) and on tasks that look unimportant until a probe
216 is actually run (description-only AUC below chance). A ranker that is honest about not being a
217 stoplight is usable for budget allocation; a non-significant AUC hidden by post-hoc features would
218 not be. The process claim, for this workshop, is narrower: a frozen protocol, a pre-registered honesty
219 clause, and fail-closed logistics were enough to publish that negative gate rather than wait for more
220 no-go tasks.

221 2.7 Interpretability

222 Protocol files, task packages, and run directories are append-only versioned (protocol_v2,
223 tasks_v3 leftover, tasks_v4 59-pack, tasks_v5 bird-only). Old versions remain on disk; a run
224 directory names the protocol and package it consumed. The ledger is append-only. Journal lines are
225 stage-boundary events; Spider is the existence proof that resume does not replay S0–S3. Harvest
226 is two-tier by design: incremental tars for monitoring (no adapters), one full tar for audit. Isolated
227 failed dirs live inside that tar. A sealed run directory is meant to be readable without the agent:
228 task.json and the package manifest, journal.jsonl stage boundaries, eval*_greedy.jsonl
229 for every labelled slice, STATUS, and metrics.json with the systems block (torch / transformers
230 / CUDA / driver / GPU name / pinned base revision / seeds / dry_run=false). If that bundle is
231 missing, the row is not a label. That is why BIRD and APPS are logistics excludes rather than no-go.

232 2.8 Limitations

233 The harness measures one base model, one LoRA recipe, greedy pass@1, and a $\Delta_{\text{full}} > 0$ gate that is
234 empirically easy (8 no-go out of 61). Cluster AUCs were pre-declared and are not estimable. BIRD
235 and APPS produced no label; we do not read that as “this model cannot learn SQL or competition-
236 style code.” Campaign GPU-hours were not summed and are not invented here. We report only
237 walls that sit in sealed reports: smoke-task S0→S6 of GSM8K ≈ 2.16 h, WinoGrande ≈ 0.51 h,
238 Spider ≈ 2.22 h effective (plus a killed half S4 ≈ 0.9 h not counted in the 2.22); a TDP-times-wall
239 upper bound of ≈ 1.5 kWh on that three-task block, not a metered reading; serial production ≈ 39 h
240 wall for 60 slots; Wave-1 TyDiQA ≈ 92 min and task419 ≈ 53 min inside the 3 h cap. The agent’s
241 connectivity misreport and the Gate 1-before-Gate 0 start are in the log because hiding them would
242 make the STOP discipline look cleaner than it was.

3 Broader Impact

Guardrails earned their keep when they failed *closed*. The BIRD checker aborted on a missing sha256 key rather than packing a substitute hash or writing a fake `metrics.json`; the later CUDA OOM likewise produced no label. GSM8K’s negative Δ_{full} came with unparseable 0/200 and a format-compliance shift from mixed `####/last-number` at base to `#### 200/200` after training — evidence against an EOS/truncation story, and the reason the operator could rule “format overwrite” instead of discarding a negative as a generation glitch. A rail that invents a number to keep a queue green is not a rail.

Pre-registration is what made the negative primary endpoint publishable. The honesty clause (bootstrap CI covering 0.5 equals a negative, to be printed) was committed before the remaining sixty GPU slots and before any AUC existed. After the seal, the tempting moves — add a feature, drop GSM8K, move the go threshold, reopen BIRD — were exactly the moves the contract forbids. With 8 no-go tasks the gate was always going to be under-powered; the clause prevented that fact from being negotiated away in the write-up.

STOP points were the human-agent interface, with a mixed record. The protocol-v3 STOP (keep 10×100 ; recitation is the probe) and the analysis STOP (no production before the SuperNI ids and the pre-reg were on the mainline) are the reason Part 1 has one recipe and one contract. The cost is real: Gate 1 started before Gate 0 was accepted, and the connectivity misreport looked like downtime until a human asked for a one-line multiplex check. A STOP that exists only as a chatbot apology does not stop a GPU.

Diagnosis was split on purpose. The agent is responsible for five-eye tables, SHA checks, journal resume, and for writing PARTIAL rather than a guessed Δ . The operator is responsible for mechanism rulings (format overwrite), for ops-fence vs measurement (APPS 3 h then 8 h), and for the exclude sentence that may be used only *before any label existed*. We would like the field to copy that split: freeze the recipe, pre-register the honesty clause, fail closed, and keep the operator on the one-way door out of the sample.

4 Disclosure Statement

Agent(s) / model(s) used The 61-row experimental table was produced by a frozen Python harness on a single serial GPU queue (protocol v2; base model Qwen2.5-1.5B-Instruct at the pinned revision in the protocol file). An LLM coding agent packed tasks, monitored the queue, diagnosed failures, executed the pre-registered analysis, and drafted Parts 1–3 of this manuscript, including figures generated from the frozen plot payload. The coding agent’s model identity is omitted here to preserve double-blind review and will be named to the program chairs. No experimental number in Part 1 was computed outside the sealed analysis script. Human status of this draft: data, hashes, and analysis artifacts were produced under the harness and the pre-registration; this text has *not* yet been line-checked by a human author. The two attestations below are therefore left unchecked.

☐ A human author has curated this work and verified every claim, figure, methodology detail, and reference.

☐ A human author takes full accountability for the submission as a human-authored, human-verified artifact.

References

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. *arXiv preprint arXiv:1911.11641*, 2019.
- Jonathan H. Clark, Eunsol Choi, Michael Collins, Dan Garrette, Tom Kwiatkowski, Vitaly Nikolaev, and Jennimaria Palomaki. TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages. *arXiv preprint arXiv:2003.05002*, 2020.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. *arXiv preprint arXiv:1903.00161*, 2019.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Long-Kai Huang, Ying Wei, Yu Rong, Qiang Yang, and Junzhou Huang. Frustratingly easy transferability estimation. *arXiv preprint arXiv:2106.09362*, 2021.
- Kevin Jamieson and Ameet Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. *arXiv preprint arXiv:1502.07943*, 2016.
- Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *arXiv preprint arXiv:1603.06560*, 2018.
- Cuong V. Nguyen, Tal Hassner, Matthias Seeger, and Cedric Archambeau. LEEP: A new measure to evaluate transferability of learned representations. *arXiv preprint arXiv:2002.12462*, 2020.
- Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

- 315 Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adver-
316 sarial winograd schema challenge at scale. *arXiv preprint arXiv:1907.10641*, 2019.
- 317 Kaichao You, Yong Liu, Jianmin Wang, and Mingsheng Long. LogME: Practical assessment of
318 pre-trained models for transfer learning. *arXiv preprint arXiv:2102.11005*, 2021.
- 319 Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li,
320 Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-
321 labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. *arXiv*
322 *preprint arXiv:1809.08887*, 2018.
- 323 Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a
324 machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and Part 1 state the ranking, registered-negative, and revision-main results with the same numbers as ANALYSIS_RESULTS_v2.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: Part 1 Primary Result and Part 2 Limitations: one base, one recipe, scarce no-go, BIRD/APPS logistics excludes, LogME/LEEP not run.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: No theorems.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Frozen protocol, pre-registration, analysis_v2.py, and per-task metrics are in the supplementary pack.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: An anonymized supplementary archive (code, metrics.json, eval.jsonl, ANALYSIS_RESULTS_v2) is provided; public upload is operator-handled.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Part 1 Method and the protocol file list model, LoRA, budgets, seeds, and decoding.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Task-level bootstrap 95% CIs and a 1000-fold label permutation; McNemar exact tests are uncorrected and labelled as such.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).

- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Part 2 reports per-stage walls and that campaign GPU-hours were not summed. Smoke-task walls are in GATE_REPORT.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: Public benchmarks only; no human subjects.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Part 3.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.

- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No dual-use generative release.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Datasets and Qwen2.5 are cited; SuperNI clone SHA is in the pre-reg supplement.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.

- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The 61-row table, protocol, and analysis scripts are documented in supplementary.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.

- 636
- 637
- 638
- 639
- 640
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
 - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

641

16. Declaration of LLM usage

642

643

644

645

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

646

Answer: [\[Yes\]](#)

647

648

Justification: The experimental base is Qwen2.5-1.5B-Instruct. An LLM coding agent ran the harness and drafted the paper (Disclosure).

649

Guidelines:

- 650
- 651
- 652
- 653
- The answer [\[N/A\]](#) means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
 - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.